

Practical - 1 HTML Concepts

HTML Concepts

1) Basic HTML structure

Every HTML file starts with a minimal structure. Save files with the `.html` extension (for example `index.html`).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My Page</title>
</head>
<body>
  <!-- Your content goes here -->
</body>
</html>
```

What each part does

- `<!DOCTYPE html>` — tells the browser this is HTML5.
 - `<html lang="en">` — root element; `lang` helps accessibility and search engines.
 - `<meta charset="utf-8">` — character encoding.
 - `<meta name="viewport" ...>` — mobile-friendly viewport settings.
 - `<title>` — text shown in the browser tab.
 - `<body>` — visible page content.
-

2) Adding CSS (three types)

CSS styles the page. There are three ways to add CSS to HTML: **inline**, **internal**, and **external**.

2.1 Inline CSS

Directly on an element using the `style` attribute. Use sparingly — it's hard to maintain.

```
<p style="color: darkblue; font-weight: bold;">This is inline-styled text.</p>
```

2.2 Internal CSS

Using a `<style>` block inside the `<head>`. Good for single-page demos or small pages.

```
<head>
  <!-- ... -->
  <style>
    .card { padding: 12px; border-radius: 6px; box-shadow: 0 1px 3px rgba(0,0,0,0.1); }
  </style>
</head>
```

2.3 External CSS (recommended)

Create a separate file (e.g. `styles.css`) and link it in the head:

```
<link rel="stylesheet" href="styles.css">
```

`styles.css`:

```
body { font-family: Arial, sans-serif; background: #f7f7fb; }
.header { text-align: center; padding: 24px; }
```

Best practices

- Prefer external CSS for maintainability.
 - Use classes (.) instead of styling elements directly when possible.
 - Keep CSS modular and organized (e.g., components, layout, utilities).
 - Use a CSS reset/normalize at the top if desired.
-

3) Adding JavaScript (three types)

JavaScript adds interactivity. There are three ways to include JS: **inline**, **internal**, and **external**.

3.1 Inline JavaScript (HTML attributes)

Attach script directly to HTML attributes like `onclick`. Use only for tiny examples.

```
<button onclick="alert('Hello!')">Click me</button>
```

3.2 Internal JavaScript (`<script>` tag)

Put a `<script>` block inside your HTML file. If you place it in `<head>` you often need `defer` or to wait for `DOMContentLoaded`.

```
<!-- at end of body (safe because DOM is loaded already) -->
<script>
  document.getElementById('demo')?.addEventListener('click', () => {
    console.log('Button clicked');
  });
</script>
```

3.3 External JavaScript (recommended)

Create `app.js` and include it with `<script src="app.js" defer></script>`.

```
<script src="app.js" defer></script>
```

app.js:

```
// runs after DOM parses because of `defer`  
document.addEventListener('DOMContentLoaded', () => {  
  console.log('Ready');  
});
```

4) HTML Lists (with examples)

Unordered list (`<ul>`)

```
<ul>  
  <li>Apples</li>  
  <li>Bananas</li>  
  <li>Cherries</li>  
</ul>
```

Ordered list (`<ol>`)

```
<ol>  
  <li>Install Node</li>  
  <li>Run server</li>  
  <li>Open browser</li>  
</ol>
```

Attributes you can use on `<ol>`: `start` (starting number), `type` (e.g., "a", "A", "i", "I", "1"), `reversed`.

```
<ol start="5" type="A" reversed>  
  <li>Step</li>  
  <li>Step</li>  
</ol>
```

Definition list (`<dl>`, `<dt>`, `<dd>`)

Used for term / description pairs.

```
<dl>
  <dt>HTML</dt>
  <dd>Markup language for web pages.</dd>
  <dt>CSS</dt>
  <dd>Style sheet language for presentation.</dd>
</dl>
```

Nested lists

You can nest lists inside list items.

```
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Orange</li>
    </ul>
  </li>
  <li>Vegetables</li>
</ul>
```

Styling list bullets

Use CSS `list-style-type` or `list-style-image` to customize bullets.

5) Displaying images (basic & responsive)

Basic `<img>`

```

```

- `alt` text is required for accessibility.
- `width` and `height` attributes provide intrinsic sizing (helps layout).

Responsive image with CSS

```
img { max-width: 100%; height: auto; display: block; }
```

srcset and **sizes** for responsive images

```

```

Browser picks the best image based on device and viewport.

The **<picture>** element (art direction)

```
<picture>  
  <source media="(max-width: 600px)" srcset="images/hero-small.jpg">  
  <source media="(min-width: 601px)" srcset="images/hero-large.jpg">  
    
</picture>
```

6) Special Characters in HTML

Certain characters in HTML have **special meanings** and must be written using HTML entities to avoid being misinterpreted by the browser.

Character	Entity code	Description
<	<	Less-than sign
>	>	Greater-than sign
&	&	Ampersand

Character	Entity code	Description
"	"	Double quote
'	'	Single quote
©	©	Copyright symbol

Example:

```
<p>5 &lt; 10 &amp;&amp; 20 &gt; 15</p>
<p>&copy; 2025 My Website</p>
```

7) Inline Frames (iframes)

An `<iframe>` allows you to embed another HTML page inside your current page.

Basic example:

```
<iframe src="about.html" width="400" height="300"></iframe>
```

Example: Navigation in one frame, content in another

index.html

```
<iframe src="nav.html" name="navFrame" width="200" height="300"></iframe>
<iframe src="chapter1.html" name="contentFrame" width="500" height="300"></iframe>
```

nav.html

```
<ul>
  <li><a href="chapter1.html" target="contentFrame">Chapter 1</a></li>
  <li><a href="chapter2.html" target="contentFrame">Chapter 2</a></li>
```

```
<li><a href="chapter3.html" target="contentFrame">Chapter 3</a></li>
</ul>
```

chapter1.html

```
<h1>Chapter 1</h1>
<p>This is the first chapter.</p>
```

Here:

- `target="contentFrame"` tells the link to load into the second iframe.
 - You can create multiple linked pages and navigate without reloading the main page.
-